

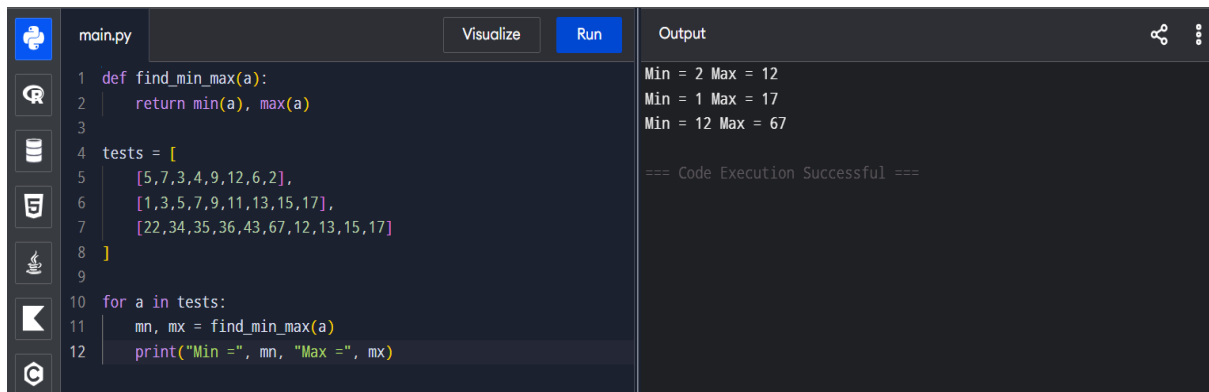
CHAPTER – 3

Name: Ramya S S

Regno: 192421226

Course: CSA0609 – DAA

EXPERIMENT – 1



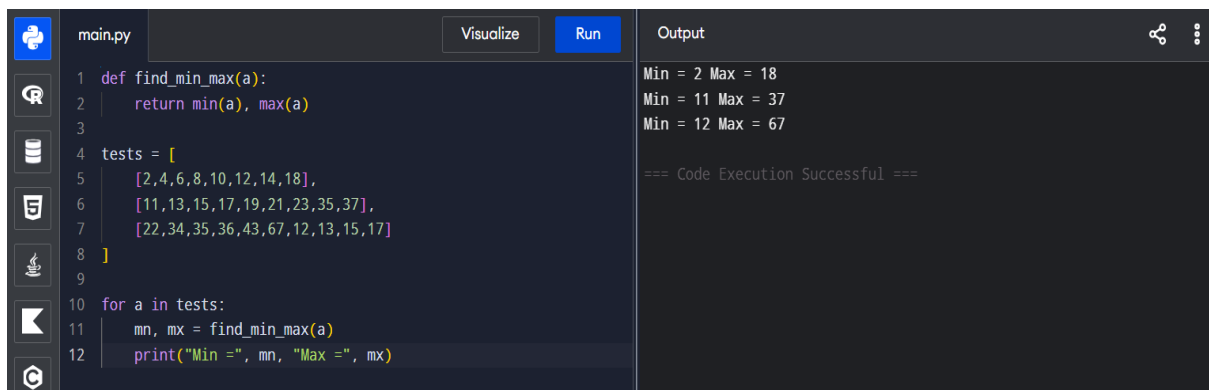
```
main.py Visualize Run
1 def find_min_max(a):
2     return min(a), max(a)
3
4 tests = [
5     [5,7,3,4,9,12,6,2],
6     [1,3,5,7,9,11,13,15,17],
7     [22,34,35,36,43,67,12,13,15,17]
8 ]
9
10 for a in tests:
11     mn, mx = find_min_max(a)
12     print("Min =", mn, "Max =", mx)
```

Output

```
Min = 2 Max = 12
Min = 1 Max = 17
Min = 12 Max = 67

=== Code Execution Successful ===
```

EXPERIMENT – 2



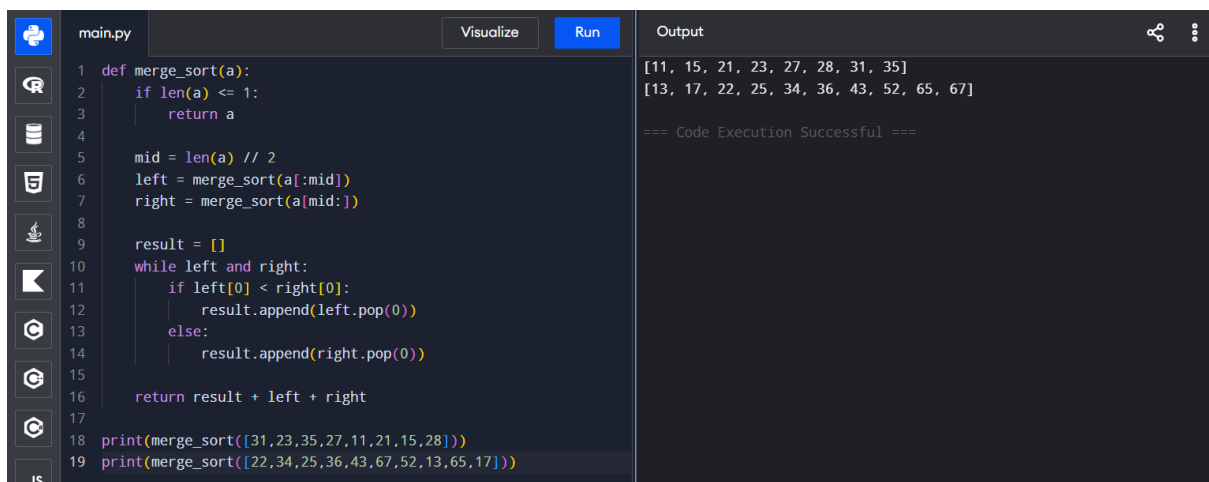
```
main.py Visualize Run
1 def find_min_max(a):
2     return min(a), max(a)
3
4 tests = [
5     [2,4,6,8,10,12,14,18],
6     [11,13,15,17,19,21,23,35,37],
7     [22,34,35,36,43,67,12,13,15,17]
8 ]
9
10 for a in tests:
11     mn, mx = find_min_max(a)
12     print("Min =", mn, "Max =", mx)
```

Output

```
Min = 2 Max = 18
Min = 11 Max = 37
Min = 12 Max = 67

=== Code Execution Successful ===
```

EXPERIMENT – 3



```
main.py Visualize Run
1 def merge_sort(a):
2     if len(a) <= 1:
3         return a
4
5     mid = len(a) // 2
6     left = merge_sort(a[:mid])
7     right = merge_sort(a[mid:])
8
9     result = []
10    while left and right:
11        if left[0] < right[0]:
12            result.append(left.pop(0))
13        else:
14            result.append(right.pop(0))
15
16    return result + left + right
17
18 print(merge_sort([31,23,35,27,11,21,15,28]))
19 print(merge_sort([22,34,25,36,43,67,52,13,65,17]))
```

Output

```
[11, 15, 21, 23, 27, 28, 31, 35]
[13, 17, 22, 25, 34, 36, 43, 52, 65, 67]

=== Code Execution Successful ===
```

EXPERIMENT – 4

main.py

Visualize

Run

1

def merge_sort(a):

2

global count

3

if len(a) <= 1:

4

return a

5

m = len(a)//2

6

left = merge_sort(a[:m])

7

right = merge_sort(a[m:])

8

result = []

9

while left and right:

10

count += 1

11

if left[0] <= right[0]:

12

result.append(left.pop(0))

13

else:

14

result.append(right.pop(0))

15

return result + left + right

16

tests = []

17

[12,4,78,23,45,67,89,1],

18

[38,27,43,3,9,82,10]

19

]

20

for a in tests:

21

count = 0

22

print("Sorted:", merge_sort(a))

Output

Sorted: [1, 4, 12, 23, 45, 67, 78, 89]

Comparisons: 16

Sorted: [3, 9, 10, 27, 38, 43, 82]

Comparisons: 13

=== Code Execution Successful ===

EXPERIMENT – 5

main.py

Visualize

Run

1

def quick_sort(a):

2

if len(a) <= 1:

3

return a

4

pivot = a[0]

5

left = [x for x in a[1:] if x <= pivot]

6

right = [x for x in a[1:] if x > pivot]

7

print("Pivot:", pivot, "->", left + [pivot] + right)

8

return quick_sort(left) + [pivot] + quick_sort(right)

9

tests = [

10

[10,16,8,12,15,6,3,9,5],

11

[12,4,78,23,45,67,89,1],

12

[38,27,43,3,9,82,10]

13

]

14

for a in tests:

15

print("Sorted:", quick_sort(a))

16

print()

Output

Pivot: 10 -> [8, 6, 3, 9, 5, 10, 16, 12, 15]

Pivot: 8 -> [6, 3, 5, 8, 9]

Pivot: 6 -> [3, 5, 6]

Pivot: 3 -> [3, 5]

Pivot: 16 -> [12, 15, 16]

Pivot: 12 -> [12, 15]

Sorted: [3, 5, 6, 8, 9, 10, 12, 15, 16]

Pivot: 12 -> [4, 1, 12, 78, 23, 45, 67, 89]

Pivot: 4 -> [1, 4]

Pivot: 78 -> [23, 45, 67, 78, 89]

Pivot: 23 -> [23, 45, 67]

Pivot: 45 -> [45, 67]

Sorted: [1, 4, 12, 23, 45, 67, 78, 89]

Pivot: 38 -> [27, 3, 9, 10, 38, 43, 82]

Pivot: 27 -> [3, 9, 10, 27]

Pivot: 3 -> [3, 9, 10]

Pivot: 9 -> [9, 10]

Pivot: 43 -> [43, 82]

Sorted: [3, 9, 10, 27, 38, 43, 82]

EXPERIMENT – 6

main.py

Visualize

Run

1

def quick_sort(a):

2

if len(a) <= 1:

3

return a

4

pivot = a[len(a)//2]

5

left = [x for x in a if x < pivot]

6

mid = [x for x in a if x == pivot]

7

right = [x for x in a if x > pivot]

8

print("Pivot:", pivot, "->", left + mid + right)

9

return quick_sort(left) + mid + quick_sort(right)

10

tests = [

11

[19,72,35,46,58,91,22,31],

12

[31,23,35,27,11,21,15,28],

13

[22,34,25,36,43,67,52,13,65,17]

14

]

15

for a in tests:

16

print("Sorted:", quick_sort(a))

17

print()

Output

Pivot: 58 -> [19, 35, 46, 22, 31, 58, 72, 91]

Pivot: 46 -> [19, 35, 22, 31, 46]

Pivot: 22 -> [19, 22, 35, 31]

Pivot: 31 -> [31, 35]

Pivot: 91 -> [72, 91]

Sorted: [19, 22, 31, 35, 46, 58, 72, 91]

Pivot: 11 -> [11, 31, 23, 35, 27, 21, 15, 28]

Pivot: 27 -> [23, 21, 15, 27, 31, 35, 28]

Pivot: 21 -> [15, 21, 23]

Pivot: 35 -> [31, 28, 35]

Pivot: 28 -> [28, 31]

Sorted: [11, 15, 21, 23, 27, 28, 31, 35]

Pivot: 67 -> [22, 34, 25, 36, 43, 52, 13, 65, 17, 67]

Pivot: 43 -> [22, 34, 25, 36, 13, 17, 43, 52, 65]

Pivot: 36 -> [22, 34, 25, 13, 17, 36]

Pivot: 25 -> [22, 13, 17, 25, 34]

Pivot: 13 -> [13, 22, 17]

Pivot: 17 -> [17, 22]

Pivot: 65 -> [52, 65]

Sorted: [13, 17, 22, 25, 34, 36, 43, 52, 65, 67]

EXPERIMENT – 7

main.py

Visualize

Run

1

def binary_search(a, key):

2

low, high = 0, len(a) - 1

3

count = 0

4

while low <= high:

5

mid = (low + high) // 2

6

count += 1

7

if a[mid] == key:

8

return mid + 1, count

9

elif key < a[mid]:

10

high = mid - 1

11

else:

12

low = mid + 1

13

return -1, count

14

tests = []

15

([5,10,15,20,25,30,35,40,45], 20),

16

([10,20,30,40,50,60], 50),

17

([21,32,40,54,65,76,87], 32)

18

]

19

for a, key in tests:

20

pos, comparisons = binary_search(a, key)

21

print("Position:", pos)

22

print("Comparisons:", comparisons)

Output

Position: 4

Comparisons: 4

Position: 5

Comparisons: 2

Position: 2

Comparisons: 2

=== Code Execution Successful ===

EXPERIMENT – 8

main.py

Visualize

Run

1

def binary_search(a, key):

2

low, high = 0, len(a) - 1

3

while low <= high:

4

mid = (low + high) // 2

5

print("Low:", low, "High:", high, "Mid:", mid)

6

if a[mid] == key:

7

return mid + 1

8

elif key < a[mid]:

9

high = mid - 1

10

else:

11

low = mid + 1

12

return -1

13

tests = []

14

([3,9,14,19,25,31,42,47,53], 31),

15

([13,19,24,29,35,41,42], 42),

16

([20,40,60,80,100,120], 60)

17

]

18

for a, key in tests:

19

print("\nArray:", a)

20

print("Position:", binary_search(a, key))

Output

Array: [3, 9, 14, 19, 25, 31, 42, 47, 53]

Low: 0 High: 8 Mid: 4

Low: 5 High: 8 Mid: 6

Low: 5 High: 5 Mid: 5

Position: 6

Array: [13, 19, 24, 29, 35, 41, 42]

Low: 0 High: 6 Mid: 3

Low: 4 High: 6 Mid: 5

Low: 6 High: 6 Mid: 6

Position: 7

Array: [20, 40, 60, 80, 100, 120]

Low: 0 High: 5 Mid: 2

Position: 3

=== Code Execution Successful ===

EXPERIMENT – 9

main.py

Visualize

Run

1

def k_closest(points, k):

2

points.sort(key=lambda p: p[0]**2 + p[1]**2)

3

return points[:k]

4

tests = []

5

([[1,3],[-2,2],[5,8],[0,1]], 2),

6

([[1,3],[-2,2]], 1),

7

([[3,3],[5,-1],[-2,4]], 2)

8

]

9

]

10

]

11

for p, k in tests:

12

print(k_closest(p, k))

Output

[[0, 1], [-2, 2]]

[[-2, 2]]

[[3, 3], [-2, 4]]

=== Code Execution Successful ===

EXPERIMENT – 10

```
main.py Visualize Run Output
1 def four_sum(A, B, C, D):
2     count = 0
3
4     for a in A:
5         for b in B:
6             for c in C:
7                 for d in D:
8                     if a + b + c + d == 0:
9                         count += 1
10
11     return count
12
13 print(four_sum([1,2], [-2,-1], [-1,2], [0,2]))
14 print(four_sum([0], [0], [0], [0]))
```

```
2
1

=== Code Execution Successful ===
```

EXPERIMENT – 11

```
main.py Visualize Run Output
1 def median_medians(a, k):
2     if len(a) <= 5:
3         return sorted(a)[k - 1]
4     groups = [a[i:i+5] for i in range(0, len(a), 5)]
5     medians = [sorted(g)[len(g)//2] for g in groups]
6     pivot = median_medians(medians, (len(medians)+1)//2)
7     low = [x for x in a if x < pivot]
8     high = [x for x in a if x > pivot]
9     equal = [x for x in a if x == pivot]
10    if k <= len(low):
11        return median_medians(low, k)
12    elif k <= len(low) + len(equal):
13        return pivot
14    else:
15        return median_medians(high, k-len(low)-len(equal))
16 print(median_medians([12,3,5,7,19], 2))
17 print(median_medians([12,3,5,7,4,19,26], 3))
18 print(median_medians([1,2,3,4,5,6,7,8,9,10], 6))
```

```
5
5
6

=== Code Execution Successful ===
```

EXPERIMENT – 12

```
main.py Visualize Run Output
1 def median_of_medians(a, k):
2     if len(a) <= 5:
3         return sorted(a)[k-1]
4     groups = [a[i:i+5] for i in range(0, len(a), 5)]
5     med = [sorted(g)[len(g)//2] for g in groups]
6     pivot = sorted(med)[len(med)//2]
7
8     low = [x for x in a if x < pivot]
9     high = [x for x in a if x > pivot]
10
11    if k <= len(low):
12        return median_of_medians(low, k)
13    elif k == len(low) + 1:
14        return pivot
15    else:
16        return median_of_medians(high, k-len(low)-1)
17
18 print(median_of_medians([1,2,3,4,5,6,7,8,9,10], 6))
19 print(median_of_medians([23,17,31,44,55,21,20,18,19,27], 5))
```

```
6
21

=== Code Execution Successful ===
```

EXPERIMENT – 13

main.py	Visualize	Run	Output
<pre>1 from itertools import combinations 2 def meet_middle(a, target): 3 n = len(a) 4 mid = n // 2 5 left = a[:mid] 6 right = a[mid:] 7 L = [] 8 R = [] 9 for r in range(len(left)+1): 10 for c in combinations(left, r): 11 L.append((sum(c), c)) 12 for r in range(len(right)+1): 13 for c in combinations(right, r): 14 R.append((sum(c), c)) 15 best = (float('inf'), ()) 16 for s1, c1 in L: 17 for s2, c2 in R: 18 diff = abs(target - (s1+s2)) 19 if diff < best[0]: 20 best = (diff, c1+c2) 21 return best[1], sum(best[1]) 22 print(meet_middle([45,34,4,12,5,2], 42))</pre>			<pre>((34, 5, 2), 41) ((4, 6), 10) === Code Execution Successful ===</pre>

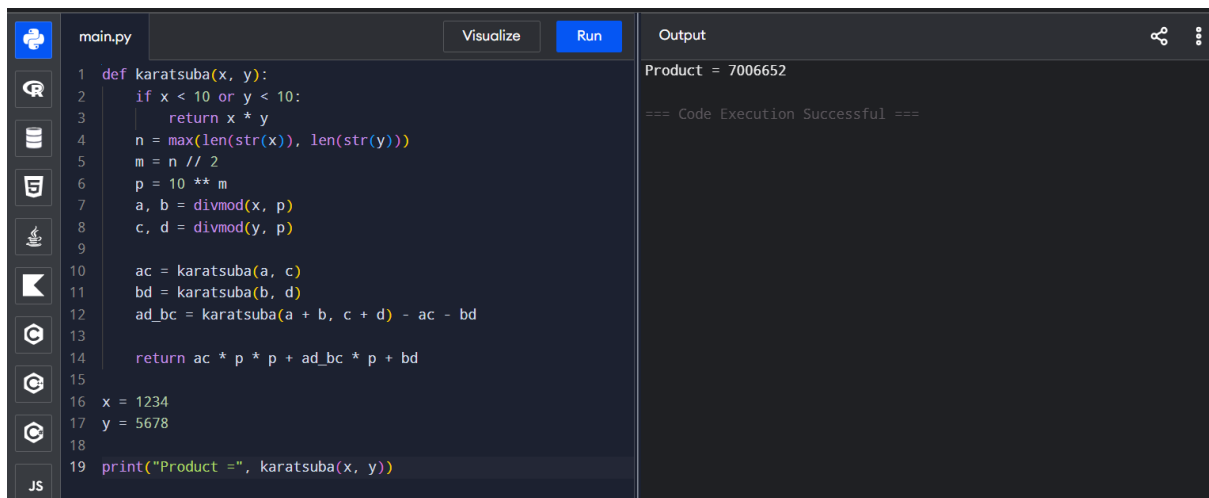
EXPERIMENT – 14

main.py	Visualize	Run	Output
<pre>1 def meet_middle(a, target): 2 n = len(a) 3 mid = n // 2 4 left = a[:mid] 5 right = a[mid:] 6 sums = set() 7 for mask in range(1 << len(left)): 8 s = sum(left[i] for i in range(len(left)) if mask & 9 (1 << i)) 10 sums.add(s) 11 for mask in range(1 << len(right)): 12 s = sum(right[i] for i in range(len(right)) if mask & 13 (1 << i)) 14 if target - s in sums: 15 return True 16 return False 17 print(meet_middle([1,3,9,2,7,12], 15)) 18 print(meet_middle([3,34,4,12,5,2], 15))</pre>			<pre>True True === Code Execution Successful ===</pre>

EXPERIMENT – 15

main.py	Visualize	Run	Output
<pre>1 def strassen(A, B): 2 a, b = A[0] 3 c, d = A[1] 4 e, f = B[0] 5 g, h = B[1] 6 p1 = a * (f - h) 7 p2 = (a + b) * h 8 p3 = (c + d) * e 9 p4 = d * (g - e) 10 p5 = (a + d) * (e + h) 11 p6 = (b - d) * (g + h) 12 p7 = (a - c) * (e + f) 13 return [14 [p5 + p4 - p2 + p6, p1 + p2], 15 [p3 + p4, p1 + p5 - p3 - p7] 16] 17 A = [[1, 7], [3, 5]] 18 B = [[6, 8], [4, 2]] 19 20 print(strassen(A, B))</pre>			<pre>[[34, 22], [38, 34]] === Code Execution Successful ===</pre>

EXPERIMENT – 16



```
1 def karatsuba(x, y):
2     if x < 10 or y < 10:
3         return x * y
4     n = max(len(str(x)), len(str(y)))
5     m = n // 2
6     p = 10 ** m
7     a, b = divmod(x, p)
8     c, d = divmod(y, p)
9
10    ac = karatsuba(a, c)
11    bd = karatsuba(b, d)
12    ad_bc = karatsuba(a + b, c + d) - ac - bd
13
14    return ac * p * p + ad_bc * p + bd
15
16 x = 1234
17 y = 5678
18
19 print("Product =", karatsuba(x, y))
```

Output

Product = 7006652

=== Code Execution Successful ===